# Experiment 'p_3_3_number_of_digits' Results

June 4, 2026

**Experiment outcome:** SUCCESS
**Bad responses:** 0
**Responses containing `assume` :** 0
**Responses containing `expect` :** 0
**Resolution attempts:** 2
**Hard fails (resolution):** 0
**Soft fails (resolution):** 0
**Verification attempts:** 2

## Problem Specification

**Problem name:** p_3_3_number_of_digits
**Natural language statement:** null
**Method signature:** p_3_3_number_of_digits (i: int) returns (n: nat)

### Ensures

- `i >= 0 ==> n == digits(i)`
- `i < 0 ==> n == digits(-i)`

### Requires

- `10000000000 > i > -10000000000`

### Functional Code Given

```
function digits (n: nat): nat
{
   if 0 <= n <= 9 then 1
   else 1 + digits (n / 10)
}
```

## GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

### Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_3_3_number_of_digits (i: int) returns (n: nat)

The method should respect the following contract:

requires 10000000000 > i > -10000000000, ensures i >= 0 ==> n == digits(i), ensures i < 0
    ↪  ==> n == digits(-i)

The contract uses the following dafny code:

function digits (n: nat): nat
{
   if 0 <= n <= 9 then 1
   else 1 + digits (n / 10)
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

### GenAI → Program

**System fingerprint:** null
**ID:** chatcmpl-Dn35i1THYrYDrTz0LVIVD3h6CD7Bg
**Model:** gpt-5.5-2026-04-23
**Created at:** 1780582326
**Finish reason:** stop
**Usage:** 428 tokens in, and 747 tokens out

```
{
  var x: int;
  if i < 0 {
    x := -i;
  } else {
    x := i;
  }

  if x < 10 {
    n := 1;
```

```
  } else if x < 100 {
    n := 2;
  } else if x < 1000 {
    n := 3;
  } else if x < 10000 {
    n := 4;
  } else if x < 100000 {
    n := 5;
  } else if x < 1000000 {
    n := 6;
  } else if x < 10000000 {
    n := 7;
  } else if x < 100000000 {
    n := 8;
  } else if x < 1000000000 {
    n := 9;
  } else {
    n := 10;
  }
}
```

### Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_3_3_number_of_digits/program.dfy(11,0): Error: a postcondition could not
    ↪ be proved on this return path
   |
11 | {
   | ^

problem_data/p_3_3_number_of_digits/program.dfy(9,22): Related location: this is the
    ↪ postcondition that could not be proved
  |
9 |  ensures i >= 0 ==> n == digits(i)
  | ^^

problem_data/p_3_3_number_of_digits/program.dfy(11,0): Error: a postcondition could not
    ↪ be proved on this return path
   |
11 | {
   | ^

problem_data/p_3_3_number_of_digits/program.dfy(10,21): Related location: this is the
    ↪ postcondition that could not be proved
   |
10 |  ensures i < 0 ==> n == digits(-i)
   | ^^


Dafny program verifier finished with 2 verified, 2 errors
```

```
Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

## GenAI → Program

**System fingerprint:** null
**ID:** chatcmpl-Dn36foJ9tz876uteDKk6XoBLpMoHf
**Model:** gpt-5.5-2026-04-23
**Created at:** 1780582385
**Finish reason:** stop
**Usage:** 1020 tokens in, and 5033 tokens out

```
{
  var x: nat;
  if i < 0 {
    assert -i > 0;
    x := -i;
  } else {
    assert i >= 0;
    x := i;
  }

  var y: nat := x;
  n := 1;
  while y >= 10
    invariant n + digits(y) == 1 + digits(x)
    decreases y
  {
    var z: nat := y / 10;
    assert z < y;
    assert !(0 <= y <= 9);
    assert digits(y) == 1 + digits(z);
    calc {
      (n + 1) + digits(z);
      ==
      n + (1 + digits(z));
      ==
      n + digits(y);
      ==
      1 + digits(x);
    }
    n := n + 1;
    y := z;
  }

  assert 0 <= y <= 9;
  assert digits(y) == 1;
  assert n + digits(y) == 1 + digits(x);
  assert n + 1 == 1 + digits(x);
```

```
  assert 1 + digits(x) == digits(x) + 1;
  assert n + 1 == digits(x) + 1;
  assert n == digits(x);

  if i >= 0 {
    assert x == i;
    assert n == digits(i);
  } else {
    assert x == -i;
    assert n == digits(-i);
  }
}
```

## Final Program

```
function digits (n: nat): nat
{
   if 0 <= n <= 9 then 1
   else 1 + digits (n / 10)
}

method p_3_3_number_of_digits (i: int) returns (n: nat)
       requires 10000000000 > i > -10000000000
       ensures i >= 0 ==> n == digits(i)
       ensures i < 0 ==> n == digits(-i)
{
  var x: nat;
  if i < 0 {
    assert -i > 0;
    x := -i;
  } else {
    assert i >= 0;
    x := i;
  }

  var y: nat := x;
  n := 1;
  while y >= 10
    invariant n + digits(y) == 1 + digits(x)
    decreases y
  {
    var z: nat := y / 10;
    assert z < y;
    assert !(0 <= y <= 9);
    assert digits(y) == 1 + digits(z);
    calc {
      (n + 1) + digits(z);
      ==
      n + (1 + digits(z));
      ==
```

```
    n + digits(y);
    ==
    1 + digits(x);
  }
  n := n + 1;
  y := z;
 }

 assert 0 <= y <= 9;
 assert digits(y) == 1;
 assert n + digits(y) == 1 + digits(x);
 assert n + 1 == 1 + digits(x);
 assert 1 + digits(x) == digits(x) + 1;
 assert n + 1 == digits(x) + 1;
 assert n == digits(x);

 if i >= 0 {
  assert x == i;
  assert n == digits(i);
 } else {
  assert x == -i;
  assert n == digits(-i);
 }
}
```

## Total Token Usage

**Input tokens:** 1448
**Output tokens:** 5780
**Reasoning tokens:** 5120
**Sum of 'total tokens':** 7228

## Experiment Timings

**Overall Experiment** started at 1780582323650, ended at 1780582555663, lasting 232013ms (232.01 seconds)
**Iteration #1** started at 1780582323677, ended at 1780582384977, lasting 61300ms (61.30 seconds)
**Iteration #2** started at 1780582384988, ended at 1780582555663, lasting 170675ms (170.68 seconds)